

PROBLEMA:	Introdução ao Diagrama de Classes	DATA:	14/08/2026
PILOTO (A):	Áleffy de Almeida	COPILOTO (A):	Augusto Brandão
QA:	Eduardo Almeida	ARQUITETO (A):	Hellen Verena
SCRUM MASTER:	Italo Mendes		

Objetivo: Modelar as entidades de negócio essenciais e iniciar o Diagrama de Classes com foco nos conceitos iniciais de Orientação a Objetos.

Fatos:

(O Scrum Master lê as perguntas. A equipe busca a resposta exata na OAT 1 e o Arquiteto documenta aqui no caderno):

1. **Atributos de Peça:** Analisando a seção "Modelagem dos Dados" da OAT, quais são os atributos obrigatórios que devem ser implementados na entidade Peça e quais são estritamente opcionais?

Fato Extraído: São atributos obrigatórios: **codigo** (Long, gerado sequencialmente pela aplicação), **nome** (String), **codigoBarras** (String), **fornecedorMarca** (String), **quantidadeEstoque** (Integer), **precoCusto** (Double), **precoVenda** (Double), **categoria** (Enum CategoriaPeca), **dataCadastro** (LocalDateTime) e **dataAtualizacao** (LocalDateTime). São atributos estritamente opcionais: **tamanho** (String) e **cor** (String).

2. **Atributos de Serviço:** De acordo com a especificação do produto na OAT, quais são os dados exatos que a entidade Serviço deve possuir para ser cadastrada validamente?

Fato Extraído: A entidade Serviço exige os seguintes dados obrigatórios: **codigo** (Long, gerado sequencialmente pela aplicação), **nome** (String), **tempoEstimadoMinutos** (Integer), **custoTabelado** (Double), **dataCriacao** (LocalDateTime) e **dataAtualizacao** (LocalDateTime).

3. **A Obrigação do Enum:** O que a diretoria determina na especificação sobre a organização do catálogo utilizando Enumerações? Qual deve ser o nome desse Enum e como ele se vincula à entidade Peça?

Fato Extraído: A especificação determina a criação obrigatória da enumeração **CategoriaPeca** com os valores constantes: **MOTOR**, **SUSPENSAO**, **FREIOS**, **ELETRICA** e **ACESSORIOS**. Esse Enum deve ser vinculado como um atributo tipado obrigatório na classe **Peca**, garantindo type safety no catálogo.

Brainstorm Investigativo:

(Questões e Ideias guiaTdas pelo Scrum Master)

1. A Modelagem Inicial: Como aplicar o encapsulamento nessas entidades para garantir a integridade dos dados? Quais modificadores de acesso usaremos para os atributos e métodos?

Resposta: Aplicamos o encapsulamento estrito definindo todos os atributos das entidades com o modificador de acesso `private (-)`, impedindo a manipulação direta do estado dos objetos. O acesso e atualização ocorrem unicamente através de métodos públicos `(+)` `getters` e `setters`. Para proteger a entidade no transporte de dados, utilizamos as classes

PecaRequestDTO e ServicoRequestDTO, impedindo que requisições externas enviem ou adulterem campos de controle interno (codigo e datas de auditoria).

2. **O Construtor:** Quais atributos devem ser obrigatórios no momento da instanciação (no construtor) e quais atributos dependerão puramente de métodos de acesso (Getters/Setters)?

Resposta: Cada entidade implementa um construtor padrão sem argumentos (`public Peca()`) para serialização/deserialização e um construtor completo parametrizado com todos os atributos para instanciação atômica no repositório. Campos gerados pelo servidor (`codigo`, `dataCadastro`, `dataAtualizacao`) são preenchidos no momento da persistência, enquanto atualizações pontuais utilizam métodos `setters`.

Cenários de Teste:

(Liderado pelo QA)

- Validar se a criação de um objeto Peça exige as categorias restritas definidas no Enum [CategoriaPeca](#).

ID do Teste: QA-ST01-01

Responsável (QA): Eduardo Almeida

O que estamos testando? (Contexto): Validar se o cadastro de uma Peça restringe a categoria exclusivamente aos valores válidos do Enum `CategoriaPeca`.

Cenário (Ação e Resultado):

Dado que o cliente envia uma requisição de cadastro de peça com a categoria `MOTOR`,

Quando a aplicação processa a atribuição dos dados no modelo de domínio,

Então o sistema deve aceitar a criação com sucesso. Caso seja informada uma categoria inexistente (ex: `PNEUS`), o sistema deve rejeitar a operação por incompatibilidade de tipo.

Status:  Especificado / Aprovado no Modelo

Observações do Piloto/Dev: Cenário de aceite planejado e validado no Diagrama de Classes UML. A validação ponta a ponta com requisições HTTP será executada na integração dos Controllers (ST3).

Entrega do Encontro:

Código: Criação de [Peca.java](#), [Servico.java](#) e do Enum [CategoriaPeca.java](#) no repositório.

- Diagrama: O Arquiteto deve iniciar o Diagrama de Classes com as entidades e o Enum preenchidos com atributos e modificadores de visibilidade.

PROBLEMA:	Introdução ao Diagrama de Classes	DATA:	14/08/2026
PILOTO (A):	Áleffy de Almeida	COPILOTO (A):	Augusto Brandão
QA:	Eduardo Almeida	ARQUITETO (A):	Hellen Verena
SCRUM MASTER:	Italo Mendes		

Objetivo: Dominar a aplicação prática de membros estáticos (static) implementando a camada de acesso a dados (Repositórios) com o Padrão de Projeto *Singleton*.

Fatos:

(O Scrum Master lê as perguntas. A equipe busca a resposta exata na OAT 1 e o Arquiteto documenta no repositório)

1. **A Restrição Arquitetural:** Segundo a introdução da OAT e a seção "Motor de Armazenamento", quais são as tecnologias e recursos do Spring Boot (relacionados a banco de dados e injeção de dependência) que são estritamente proibidos nesta fase inicial?

- **Fato Extraído:** É estritamente proibido o uso de bancos de dados físicos ou embarcados (PostgreSQL, MySQL, H2), interfaces do Spring Data JPA e o uso de injeção de dependência gerenciada pelo Spring Boot (@Autowired, @Component, @Repository) na camada de dados. Toda a persistência é mantida 100% em memória RAM através de coleções Java (List).

2. **O Padrão de Armazenamento:** A OAT exige a criação de repositórios manuais (ex: [PecaRepository](#)). Qual Padrão de Projeto deve ser obrigatoriamente utilizado para implementá-los e qual método deve conceder acesso exclusivo a eles?

- **Fato Extraído:** Os repositórios devem implementar obrigatoriamente o Padrão de Projeto de Criação Singleton manual, cujo acesso à instância única da classe é concedido exclusivamente através do método público e estático `getInstance()`.

Brainstorm Investigativo:

(Questões e Ideias guiadas pelo Scrum Master)

- **O Construtor Privado:** Por que o construtor dessas classes repositório precisa ser *private*? O que isso impede que os programadores façam no futuro?

Resposta: O construtor é definido como *private* para bloquear a instanciação direta fora da classe, impedindo que desenvolvedores criem novos objetos usando `new PecaRepository()`. Isso garante que toda a aplicação compartilhe uma única instância global na memória, prevenindo a criação de repositórios paralelos e a perda de dados entre requisições.

- **A Sobrevivência dos Dados:** Como as listas internas de persistência vão sobreviver e se manter as mesmas enquanto o servidor estiver rodando? Qual é o papel do atributo *static* nesse comportamento?

Resposta: O modificador `static` no atributo `private static PecaRepository`

`INSTANCE` vincula a referência da instância única à classe carregada na memória da JVM (nível de classe) e não a instâncias voláteis na memória heap. Dessa forma, as listas internas (`List`) persistem durante todo o ciclo de vida da aplicação.

Cenários de Teste:

(Liderado pelo QA)

- Testar se o programa falha em tempo de compilação ao tentarmos instanciar o `PecaRepository` usando a palavra-chave `new` fora da própria classe.

ID do Teste: QA-ST02-01

Responsável (QA): Eduardo Almeida

O que estamos testando? (Contexto): Validar o bloqueio de instanciação externa direta do Repositório via operador `new`.

Cenário (Ação e Resultado):

Dado que a classe `PecaRepository` possui um construtor privado,

Quando tentamos instanciar um novo objeto (`new PecaRepository()`) de outra classe,

Então o compilador Java deve gerar um erro de acesso, garantindo que o Singleton seja o único ponto de entrada.

Status:  Especificado / Aprovado no Modelo

Observações do Piloto/Dev: Cenário de aceite planejado e validado no Diagrama de Classes UML. A validação ponta a ponta com requisições HTTP será executada na integração dos Controllers (ST3).

Entrega do Encontro:

- **Código:** As classes `PecaRepository.java` e `ServicoRepository.java` implementando o padrão Singleton de forma manual, contendo as listas internas.

- **Diagrama:** Atualizar o Diagrama de Classes, evidenciando o sublinhado em membros UML que indica que os atributos e métodos `getInstance()` são estáticos.

PROBLEMA:	Introdução ao Diagrama de Classes	DATA:	28/08/2026
PILOTO (A):	Italo Mendes	COPILOTO (A):	Áleffy de Almeida
QA:	Augusto Brandão	ARQUITETO (A):	Eduardo Almeida
SCRUM MASTER:	Hellen Verena		

Objetivo: Mapear rotas na Web API estilo REST, compreender as anotações do framework Spring Boot e estabelecer os relacionamentos lógicos entre a interface web e os repositórios.

Fatos:

(O Scrum Master lê as perguntas. A equipe busca a resposta exata na OAT 1 e o Arquiteto documenta no repositório)

1. **Rotas e Controladores:** De acordo com a seção "Roteamento REST e Controladores" da OAT, qual é o padrão estrutural que as rotas da nossa aplicação devem seguir para expor os dados das peças e serviços?
 - **Fato Extraído:** As rotas devem seguir o padrão arquitetural RESTful estruturadas nas URLs base `/api/pecas` e `/api/servicos`, utilizando os métodos HTTP semânticos (POST, GET, PUT, DELETE) para as operações de CRUD.
2. **A Semântica de Sucesso:** A OAT determina que a API não pode simplesmente devolver os dados puros. Quais são os Status HTTP exatos que devem ser retornados para Criação com sucesso, Listagens/Atualizações, e Deleções bem sucedidas?

- **Fato Extraído:** A API deve retornar respostas semânticas encapsuladas em ResponseEntity: 201 Created para criação bem-sucedida (POST); 200 OK para consultas e atualizações (GET/PUT); e 204 No Content para exclusões bem-sucedidas (DELETE).

3. **A Semântica de Erro:** Qual Status HTTP a OAT exige que seja codificado e retornado caso uma operação de busca ou deleção não encontre o ID solicitado pelo cliente?

- **Fato Extraído:** Operações de busca (GET), atualização (PUT) ou deleção (DELETE) que não localizem o identificador (codigo) solicitado devem retornar obrigatoriamente o status HTTP 404 Not Found.

Brainstorm Investigativo:

(Questões e Ideias guiadas pelo Scrum Master)

- **Integração:** Como faremos com que o Controller peça para o Repository salvar o dado, lembrando que a OAT proíbe a injeção de dependência padrão do Spring?

Resposta: A integração é realizada via acesso estático ao Singleton. No Controller, acessamos a instância do repositório chamando `PecaRepository.getInstance()`, o que nos permite invocar métodos de manipulação de lista (salvar, buscar, remover) sem violar as restrições arquiteturais.

- **Relacionamentos no Diagrama:** Como representaremos no Diagrama de Classes a relação visual entre o `PecaController` e o `PecaRepository`?

Resposta: No Diagrama de Classes, representamos o Controller como uma classe que possui uma associação de uso (dependência) com o Singleton Repository. A relação indica que o Controller invoca o método estático `getInstance()` do Repository, demonstrando o fluxo de controle da camada web para a camada de persistência.

Cenários de Teste:

(Liderado pelo QA)

ID do Teste: QA-ST03-01

Responsável (QA): Hellen Verena

O que estamos testando? (Contexto): Validar a criação de recursos via API com retorno **201 Created** e preenchimento de campos gerados.

Cenário (Ação e Resultado):

Dado que o cliente envia uma requisição POST /api/pecas com JSON válido,

Quando o PecaController processa a inclusão via repositório,

Então o status HTTP 201 Created deve ser retornado contendo a Peça com código e datas geradas.

Status:  Passou / Aprovado no Postman Runner

Observações do Piloto/Dev: Executado e validado com sucesso via Collection oficial (mecaniQA-sao-luiz.postman_collection.json) com retorno 201 Created.

ID do Teste: QA-ST03-02

Responsável (QA): Hellen Verena

O que estamos testando? (Contexto): Validar o retorno manual de 404 Not Found ao tentar buscar uma Peça cujo ID ainda não foi cadastrado no Singleton.

Cenário (Ação e Resultado):

Dado que o cliente realiza uma requisição GET /api/pecas/9999 para um ID inexistente no repositório,

Quando o repositório retornar Optional.empty(),

Então o **PecaController** deve responder com status **HTTP 404 Not Found** de forma semântica.

Status:  Passou / Aprovado no Postman Runner

Observações do Piloto/Dev: Tratamento com **Optional** e

ResponseEntity.notFound().build() validado com sucesso no Postman Runner.

Entrega do Encontro:

- **Código:** [PecaController.java](#) e [ServicoController.java](#) com endpoints implementados para suprir as 8 User Stories.
- **Artefatos:** *Collection* (Postman ou Insomnia) exportada com as rotas criadas. O Arquiteto finaliza o Diagrama de Classes.